

SRIVIN PLATFORMS

Delivery partner to RacingAhead / SportSeek

SportSeek — Unified Sports Ecosystem Platform

Technical Approach

How Srivin Platforms will engineer SportSeek Phase 2A: a unified, compliant and scalable sports platform built on what is already live

Document control	
Document	SRV-SPS-2A-TAD-003
Programme	SportSeek Phase 2 — Phase 2A (Harden & Unify)
Client	RacingAhead / SportSeek, Hyderabad
Prepared by	Srivin Platforms — Technical Programme Management Office
Version / date	v2.0 (AI-native release train) — 27 September 2026
Status	Issued for client review — supersedes v1.0
Reference baseline	SportSeek Phase 2 SOW v1.0 (15 Jul 2026); SportsSeek App High Level Requirements
Classification	Confidential — RacingAhead / SportSeek and Srivin Platforms only

Contents

- 1. Executive summary
- 2. The problem, technically
- 3. Architecture principles
- 4. Starting from what is live: Phase 1 assessment
- 5. Target architecture
- 6. Module-by-module technical approach
- 7. Cross-cutting quality attributes (NFRs)
- 8. Engineering practices and standards
- 9. AI-accelerated engineering
- 10. Technical plan for the 20 October 2026 launch
- 11. Data migration approach
- 12. Key technical decisions (ADR backlog)
- 13. Technical risks and mitigations
- 14. Why Srivin Platforms
- Appendix A. Glossary

1. Executive summary

SportSeek has something most platforms at this stage do not: a live product with real users, partners and payments. Phase 2A is not a greenfield build. It is a careful re-foundation of a running system. SportSeek must go from "facility booking with separate app accounts" to "one identity, one partner model, one transaction view" without breaking the journeys users rely on today, and while going public on 20 October 2026.

Our technical approach is built around that reality:

1. **Harden, then unify, then extend, one month at a time.** We make the live product safe for public traffic and add two features on day one (R1, 20 Oct 2026). Then two features reach production every month, in dependency order: identity (R2), refunds and the player web app (R3), the generalised partner model (R4), the Ledger and partner web (R5), and payouts and WhatsApp (R6, 23 Mar 2027).
2. **A .NET modular monolith with hard domain boundaries** on PostgreSQL, extending SportSeek's confirmed React Native / .NET / PostgreSQL stack, as the SOW recommends (SOW §6). A small team moves fast in a single deployable, while each domain has its own schema, public interface and events. Any domain can be extracted into a service later without a rewrite.
3. **Identity designed against the one failure that matters:** duplicate accounts across the User and Partner apps (SOW §7.3). Account linking is verified by OTP ownership proof, existing duplicates are reconciled with rules SportSeek approves, and the linking behaviour is an automated acceptance test.
4. **Payments stay gateway-native.** The Transaction Ledger is a read model reconciled against Razorpay, and partner payouts use Razorpay Route split settlement, so SportSeek never holds or routes funds (SOW §8.3).
5. **Configuration, not code, for new partner types.** Service types are data, with schemas an admin can enable (SOW §9.4). Phase 2B's expert booking then becomes an extension, not a new system.
6. **AI-accelerated, human-owned engineering.** AI coding agents build from approved specifications, generate tests and keep documentation current. Senior engineers review and own every change, and the same automated tests, security scans and performance tests apply to everything (Section 9).

The outcome for SportSeek

A public launch on 20 October 2026 on a security-tested, load-tested and instrumented product, with two new features on day one. Two more features every month after that, and Phase 2A complete on 23 March 2027, delivered by a lean AI-native pod. A codebase, documentation and SportSeek engineering team able to carry Phase 2B and 2C forward.

2. The problem, technically

We read the SOW for the engineering problems underneath the feature list. These six determine whether Phase 2A succeeds:

Challenge	Why it is hard	Our answer (details in Section 6)
Unifying identity in a live system	Phase 1 users may already have separate records per app. Linking on phone/email alone is an account-takeover risk. Old app versions in the field still call old APIs.	One Identity service with an OIDC client per app; OTP-verified linking; duplicate reconciliation with approved merge rules; versioned APIs and minimum-version enforcement.
Generalising the partner model without regressions	Facilities are live and bookable. Changing their data model risks breaking bookings and partner payouts.	Shared service core plus type-specific attributes validated by per-type schemas; migrate facilities as the first type behind a feature flag; full facility regression pack.
A transaction view with no money held	The ledger must match the gateway exactly, stay current, and avoid any design that makes SportSeek a Payment Aggregator.	Read model from Razorpay webhooks plus scheduled reconciliation; the gateway reference is the unique key; Route split settlement; no balances anywhere.
Hardening bookings under concurrency	Cancellation, refunds and waitlists add state transitions to flows that must never double-book or double-refund.	Database-level overlap constraints, idempotency keys on payment and refund commands, explicit booking state machine, waitlist offers with expiry.
Multi-channel notifications in India	SMS needs registered DLT templates; WhatsApp needs pre-approved templates and opt-in; Push needs device tokens per app; alerts must respect role scoping.	Event-driven notification service; catalog and templates as managed configuration; per-role preferences; channel fallback; delivery tracking.
Web parity for everything mobile does	Every Phase 1 journey plus every 2A feature must work on the web, stay consistent with mobile, and not fork business logic.	One API surface through the BFF; shared design system; parity matrix as an acceptance artefact; public event pages built for search visibility.

3. Architecture principles

#	Principle	Consequence for design
1	Domains own their data	Each domain has its own Postgres schema (SOW §6.1). No cross-schema joins or writes. Other domains use its public module interface or its events.
2	Events for cross-domain side effects	"Booking confirmed → notify + record analytics" runs through the event bus, not chained synchronous calls (SOW §6.1). Events are published through a transactional outbox so they are never lost or sent for rolled-back work.
3	The gateway is the system of record for money	SportSeek stores references and states, never balances or card/UPI credentials (SOW §8).
4	Backward compatibility is a requirement	Old mobile clients in the field keep working until they are moved on with minimum-version enforcement. APIs are versioned and database migrations are expand-then-contract.
5	Configuration over code for variability	Service types, alert catalog, templates, refund rules and feature flags are data managed from admin tooling.
6	Secure and private by default	OWASP-aligned controls, least privilege, encryption in transit and at rest, and DPDP-aligned data handling (SOW §6.2).
7	Observable from day one	Structured logs, metrics, traces and business health signals (e.g. payments captured per minute) exist before a feature is called Done.
8	Buy the commodity, build the differentiator	Use managed services and off-the-shelf tools for infrastructure, notifications delivery and BI (SOW §6.1 Analytics). Build SportSeek's domain logic.

4. Starting from what is live: Phase 1 assessment

SportSeek has confirmed the Phase 1 technology stack: **React Native** mobile apps, a **.NET** back end and a **PostgreSQL** database. Phase 2A builds on that stack rather than replacing it (SOW §6). Knowing the stack is not the same as knowing the codebase, so our first technical act is a structured assessment of the live system. A launch-focused pass runs over 3 days (29 Sep – 1 Oct) and a full deep-dive over 2 weeks (5 – 16 Oct).

Area	What we examine	Output
Architecture & code	.NET and React Native versions and support status, solution/module structure, code quality, NuGet and npm dependency health, test coverage, build process	Architecture as-is; technical-debt and upgrade register
Data	PostgreSQL schema and migration history, identity/user tables per app, facility/service model, event/tournament model, indexes, data quality, duplicates	ER diagram as-is; duplicate-identity estimate
Integrations	Payment gateway integration (flows, webhooks, refunds), SMS provider, maps/location	Integration inventory and gaps vs 2A
Security	Authentication and session handling, authorisation checks, secrets, dependency vulnerabilities, OWASP Top 10 exposure	Security findings, prioritised for launch
Infrastructure & operations	Environments, CI/CD, backups and restore, monitoring, logging, scaling limits, cost	Operational readiness gaps
Mobile release posture	Store accounts, signing, version distribution, forced-upgrade capability, store policy compliance	Release-readiness checklist

The assessment feeds two decisions: the launch scope agreed at the 1 October Scope Gate, and the HLD choices in Section 11. Where Phase 1 is sound we build on it, as the SOW asks (SOW §6); where it is not, we say so with evidence and options.

5. Target architecture

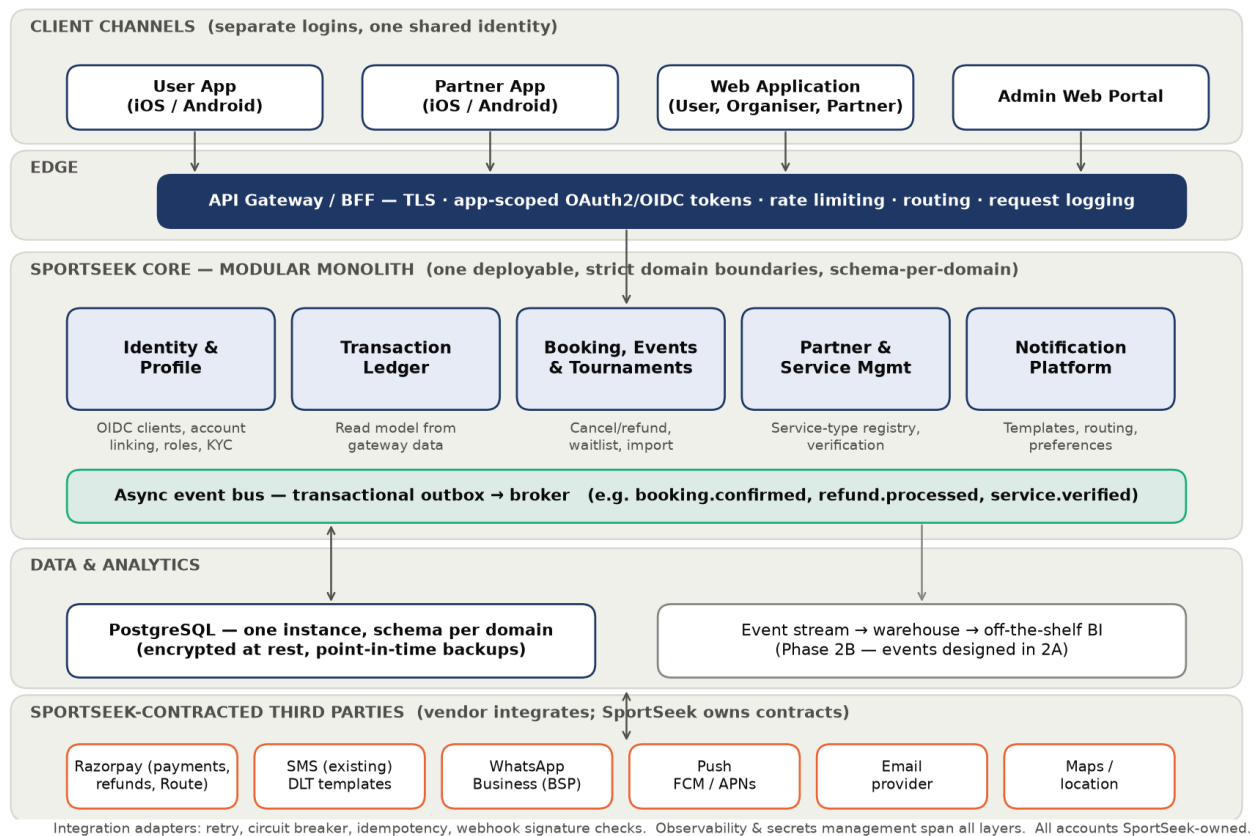


Figure 1 — Phase 2A target architecture (logical view)

5.1 Layers

Layer	Design
Client channels	User App and Partner App (iOS/Android), the new Web Application (User, Organiser, Partner journeys) and the Admin Web Portal. Each keeps its own login, as the SOW requires (SOW §7), and is registered as its own OAuth2/OIDC client against the shared Identity service.
API Gateway / BFF	Single entry point: TLS, token validation, app-scoped tokens, rate limiting, request correlation IDs, routing. Thin client-specific BFF endpoints shape responses for mobile and web without duplicating business logic.
Core domains (modular monolith)	Identity & Profile; Transaction Ledger; Booking, Events & Tournaments; Partner & Service Management; Notification. One deployable with enforced module boundaries. Architecture tests in CI fail the build if one module reaches into another's internals or schema.
Async event bus	Transactional outbox in each domain's schema, relayed to a managed message broker in SportSeek's cloud. Consumers are idempotent. Failed messages go to a dead-letter queue with alerting.
Data	One PostgreSQL instance with a schema per domain (SOW §6.1); encryption at rest; automated backups with point-in-time recovery; read replica if reporting load requires it.
Analytics readiness	Domain events follow a documented taxonomy from 2A. Phase 2B can then stream them into a warehouse and an off-the-shelf BI tool (SOW §6.1, §9.9) without re-instrumenting.
Third parties	All SportSeek-contracted (SOW §15). Each sits behind an adapter with timeouts, retries with back-off, circuit breaking, idempotency and webhook signature verification.

5.2 Why a modular monolith, and when to split

The SOW recommends a modular monolith over "7+ independently deployed microservices, so a small team can move fast" (SOW §6). We agree, for concrete reasons: a single deployment pipeline, transactional consistency

inside a domain, simpler local development for SportSeek developers joining the team, and lower infrastructure cost. The boundaries are what matter, and we enforce them in code and in CI. A domain is extracted into its own service only when there is a measured reason: independent scaling needs, a separate team owning it, or a materially different availability requirement. The schema-per-domain and event-driven integration make that extraction mechanical, not a rewrite.

5.3 Technology stack

Layer	Technology (confirmed / proposed)	How we use it in Phase 2A
Mobile apps	React Native with TypeScript (confirmed)	Extend the existing User and Partner apps: OIDC with PKCE (e.g. react-native-app-auth), push via FCM/APNs (e.g. React Native Firebase), deep links and app-switch handling, schema-driven onboarding forms, minimum-version enforcement; builds and store releases automated with Fastlane
Back end	.NET — ASP.NET Core Web API, C# (confirmed)	One modular-monolith solution with a project per domain module; module boundaries enforced by architecture tests (NetArchTest or ArchUnitNET); hosted background services for outbox relay, reminders and reconciliation
Data access	EF Core with the Npgsql provider (proposed, subject to the Phase 1 assessment)	A DbContext per domain mapped to its own PostgreSQL schema; migrations per module; raw SQL where PostgreSQL features need it (exclusion constraints, JSONB indexes)
Database	PostgreSQL (confirmed)	Schema per domain; JSONB for service-type attributes; exclusion constraints against double-booking; geospatial radius search; point-in-time recovery
Identity	OIDC server within the .NET estate (ADR-01)	See Section 6.1
Web application	React + TypeScript with Next.js (proposed, ADR-05)	Same language and design tokens as the React Native apps; server rendering for public, shareable event pages
Testing	xUnit, Testcontainers for .NET (real PostgreSQL in tests), Jest + React Native Testing Library, Detox or Maestro, Playwright, k6 or JMeter	The test pyramid in Section 8, running in CI
Observability	OpenTelemetry for .NET; centralised logging and error tracking; mobile crash reporting	Section 7.4

6. Module-by-module technical approach

6.1 Shared Identity & Profile (F3 — R2, 24 Nov 2026)

Identity model

- **Identity:** one record per person; the anchor for everything else.
- **Verified contacts:** phone (normalised to E.164) and email (normalised), each with a verified flag and timestamp; uniqueness enforced on verified values.
- **Credentials:** one credential set per identity. A password or credential change applies wherever the person logs in (SOW §7.4).
- **Role assignments:** Player, Event Organiser, Facility Partner, Coach, Physio, Nutritionist and Admin roles held on one identity (SOW §7.2).
- **KYC / verification:** attached to the identity and reused across roles (SOW §7.4); status and evidence references stored, with sensitive documents in restricted storage.
- **Role-scoped data and preferences:** notification preferences and role-specific profile data live per role, so a Player never receives facility-owner alerts (SOW §7.4).

Authentication

- Each app (User, Partner, Admin, Web) is its own OIDC client. Tokens are app-scoped and carry the roles relevant to that app. There is no silent session sharing, because SSO is out of scope (SOW §4, §7).
- Mobile and web use Authorization Code with PKCE. Access tokens are short-lived; refresh tokens rotate and can be revoked per device.
- OTP flows are rate-limited per phone, device and IP, with attempt caps and lockout, to resist brute force and SMS pumping.

Account linking (SOW §7.3), the flow we design against duplicates

1. A person registers on the Partner App (or User App) with a phone number or email.
2. The Identity service normalises the identifier and looks up **verified** contacts.
3. If a match exists, the person must **prove ownership by OTP** on that contact. Linking on a matching number alone would let anyone claim someone else's account.
4. On successful proof, the new role is **added to the existing identity**. No second record is created; profile and KYC become visible from both apps (SOW §9.1).
5. If no identity exists, the person is guided to register, and the app they register from determines the initial role (SOW §7.3).
6. Every linking decision is audit-logged. An automated acceptance test asserts "existing User App phone registers on Partner App → same identity, role added, no duplicate".

Existing data and backward compatibility

Phase 1 may already contain people with separate User and Partner records. We run a duplicate-detection report early (WBS 5.7). SportSeek approves the merge rules (D18): which profile fields win, and how bookings, events and payouts are re-pointed. The migration is rehearsed on Staging with full reconciliation counts before production. Old app versions keep working through versioned endpoints until minimum-version enforcement moves users onto the updated apps.

Build or adopt the identity provider

Option	Strengths	Watch-outs
ASP.NET Core Identity + OpenIddict (open-source OIDC server library for .NET), inside the SportSeek back end	Same stack and team skills as Phase 1; no per-user licence; account-linking, roles and KYC logic sit next to the identity data	SportSeek owns security patching of the library; OIDC configuration must be security-reviewed
Standalone open-source identity server (e.g. Keycloak), self-hosted	Standards-complete OIDC and admin console out of the box	Separate Java-based runtime to operate; custom linking flows need extensions outside the .NET codebase
Managed identity service from SportSeek's cloud provider or a vendor	Least operations; mature security features	Per-user cost at scale; OTP/phone flows and custom linking logic need checking against the provider's limits

Recommendation: given the confirmed .NET stack, our leaning is ASP.NET Core Identity with OpenIddict. It keeps identity in the same codebase, language and deployment as the rest of the platform, and SportSeek's .NET developers can own it after KT. The final choice is made in the HLD (ADR-01) after the Phase 1 assessment reviews the current authentication code. Whichever option is chosen, the account-linking logic, role model and KYC reuse sit in SportSeek's Identity domain, so the provider stays replaceable.

6.2 Notification Platform (F1, F2 — R1; F4 — R2; F12 — R6)

- **Event-driven:** domains publish business events such as booking.confirmed, event.updated, waitlist.slot_opened and refund.processed. The Notification service maps events to catalog entries; domains never call channel providers directly.
- **Catalog as configuration:** each trigger defines its recipients (by role), channels, template per channel and language, and priority. It starts from SOW §10 and is extended jointly during LLD (SOW §10.3 ▀).
- **Template management:** versioned templates with preview and test-send from admin tooling. Templates and content are supplied by SportSeek (SOW §10).
- **Channels:** SMS through the existing provider, with new commercial templates registered on the DLT platform as Indian regulation requires. Push through FCM/APNs with device tokens per app and per identity. WhatsApp through SportSeek's Business Solution Provider, using Meta-approved templates and recorded opt-in. Email through SportSeek's provider with authenticated sending domains (SPF/DKIM/DMARC).
- **Preferences & scoping:** per role and per channel; mandatory transactional and security alerts cannot be switched off; opt-in alerts (e.g. new events matching followed sports) respect consent.
- **Reliability:** idempotent sends (one message per event × recipient × channel), retries with back-off, fallback (e.g. Push then SMS for time-critical reminders), and a delivery-status log with provider callbacks.
- **Deep links across apps:** a notification that needs an action in the other app uses explicit app-switch handling (open the other app, or its store page if it is not installed), not a shared session (SOW §7.4).
- **Scheduled reminders** (T-24h, T-2h, per SOW §10.2) come from a durable scheduler and are cancelled or re-timed automatically when bookings or events change.

6.3 Partner & Service Management — generalised (F7 — R4; payouts F11 — R6)

- **Service core:** name, description, pricing, availability, location, verification status, owning partner (SOW §9.4 shared fields).
- **Service types as data:** each type (Facility, Coach, Physio, Nutritionist, and future types) has a versioned attribute schema, e.g. amenities for facilities, certifications and session format for coaches. Type-specific attributes are stored as validated structured documents (Postgres JSONB checked against the type's schema). Admins enable a type and its schema from the dashboard with no code change (SOW §9.4 acceptance).
- **Dynamic onboarding forms** in the Partner App and web are rendered from the type schema, so a new type needs no app release unless it brings a new interaction pattern.

- **Verification workflow per type:** a configurable checklist (documents, certifications) reviewed in the Admin Portal, with the status change published as an event that triggers notifications (SOW §10.2).
- **Facilities migrate first:** existing facilities become type "Facility" through a scripted, reversible migration behind a feature flag, backed by a full facility booking and payout regression. This is SOW §9.4's criterion that "existing facility partners are unaffected".
- **Payout onboarding:** partner KYC and bank fields needed by Razorpay Route are collected at onboarding and passed through to create linked accounts (SOW §8.3). SportSeek relays the data; the gateway runs KYC checks for payouts.
- **Ready for 2B:** availability and booking for any service type use the same engine, so the "Expert Booking Experience" is a user-experience layer on this model, not a separate marketplace (SOW §3.2, §9.6).

6.4 Booking, Events & Tournaments hardening (F5 — R3; F8 — R4)

- **No double-booking by construction:** slot reservations use database-level overlap protection (a PostgreSQL exclusion constraint on resource and time range) plus short-lived holds during checkout. This does not rely on application checks alone.
- **Explicit state machines** for bookings and registrations (held → confirmed → cancelled → refund pending → refunded, etc.). Transitions are validated centrally and every transition emits an event.
- **Cancellation & refund:** a policy engine evaluates SportSeek's rules (cut-off times, percentages, non-refundable items, who approves) from configuration (D11). Refunds go through the gateway refund API to the original payment method with an idempotency key, so a retry never double-refunds (SOW §8.2).
- **Waitlist:** first-in, first-out queue per slot or tournament. When capacity frees up, the next person receives a time-boxed offer by Push/SMS; unclaimed offers roll to the next person automatically (SOW §10.2).
- **Edit-after-publish:** material changes to a published event (date, venue, format) create an "event updated" event that notifies all registered users, with the changed fields shown (SOW §9.3, §10.2).
- **Consistent data model:** events and tournaments reference services from the generalised Partner & Service model, so later service types can host events without schema forks (SOW §9.3).
- **Import tools:** an offline command-line importer for completed-event data (SOW §9.3) using agreed CSV templates, schema validation, a dry-run report and idempotent re-runs.
- **Regression guarantee:** the automated Phase 1 regression suite, built for launch, runs on every change. SOW §9.3 requires "all Phase 1 flows continue to work unchanged".

6.5 Transaction Ledger (F9 — R5; settlement view F11 — R6)

- **What it is:** a read-side view of payment activity, sourced from gateway records. It is not a wallet: no balances, credits or refund-to-credit (SOW §8).
- **Ingestion:** Razorpay webhooks (payment, refund, transfer and settlement events) are verified by signature and processed idempotently. Scheduled reconciliation jobs pull from the gateway's APIs and reports to catch anything a webhook missed, so the ledger converges to the gateway within the agreed sync window (SOW §8.2).
- **Data model:** one row per gateway transaction, keyed on the gateway reference ID. It links to the SportSeek booking, registration or service reference, the identity and the partner. Amounts are stored as integers in the smallest currency unit to avoid rounding errors.
- **Screens (SOW §8.2):** Transaction History with filters by date range, service type, partner and status, and sorting on every column; Transaction Detail; Statement Export (PDF/CSV); Refund Status Tracker; and a partner settlement view in the web application sourced from settlement reports.
- **Accuracy guarantee:** exports are generated from reconciled ledger rows. A daily reconciliation report flags any mismatch against the gateway, and a mismatch is treated as a defect. This directly supports the SOW criterion that the "statement export matches gateway records exactly".

- **Visibility:** Admin dashboard plus a partner settlement view on web, as SOW §8.1–8.2 describe. Whether users and partners also need ledger views in the mobile apps is an open SOW item (■, D15), and the ledger APIs are designed so those views can be added without rework.

Compliance by architecture (SOW §8.3)

RBI Payment Aggregator boundary: funds flow from payer to partner through Razorpay Route split settlement, never through a SportSeek-controlled account. We treat this as a hard architectural constraint and will not design any flow that parks funds with SportSeek.

PCI-DSS scope minimisation: card and UPI credentials are only ever handled by the gateway's SDK or checkout. SportSeek systems store tokens and references only.

GST: SportSeek's own commission or convenience fee is modelled as a separate revenue line so it can be invoiced separately from the underlying booking payment.

DPDP Act: the ledger and identity data SportSeek stores follows the privacy controls in Section 7.2.

6.6 Web Application (F6 players — R3; F10 organisers & partners — R5)

- **Scope:** all existing mobile functionality for users, event organisers and venue partners (SOW §3.1), plus each Phase 2A feature as it ships, and the partner settlement view (SOW §8.2).
- **One back end:** the web app uses the same APIs through the BFF. There is no duplicated business logic, so mobile and web cannot drift.
- **Parity matrix:** every mobile journey is listed with its web equivalent and status. The signed matrix is the acceptance artefact for "web application for all the existing functionality".
- **Framework (ADR-05):** we propose **React with TypeScript on Next.js**. It uses the same language and React component model as the React Native apps, so SportSeek's developers and our shared design tokens work across both. It also provides server rendering for public, shareable pages such as event and tournament listings, which benefit from search visibility. We considered React Native for Web to maximise code sharing and prefer Next.js for SEO and web-native UX; the ADR records the trade-off.
- **Responsive and accessible:** layouts from mobile to desktop; accessibility checked against WCAG 2.1 AA criteria for key journeys; tested on current Chrome, Safari, Edge and Firefox.

6.7 Design system and UX (all releases)

The SOW asks for UI/UX following atomic design principles, with a uniform colour and typography system approved before development (SOW §13, §14.2). We produce one design system of design tokens, atoms, molecules, organisms and templates. It serves User, Partner, Admin and Web, so the unified platform also looks unified. Discovery includes short research sessions with players, organisers and partners to validate the new flows (account linking, role switching, cancellation, waitlist). Those are the moments where confusion creates support load.

7. Cross-cutting quality attributes (NFRs)

7.1 Security (SOW §6.2)

Control area	Approach
Standards	OWASP Top 10 for web and APIs; OWASP ASVS as the verification checklist; OWASP MASVS for the mobile apps.
Identity & access	OIDC with PKCE; short-lived tokens; refresh-token rotation; RBAC enforced server-side on every endpoint; admin actions audit-logged.
Data protection	TLS everywhere; encryption at rest for databases, backups and object storage; secrets in a managed secrets store; KYC documents in restricted storage with access logging.
Application security	Input validation, output encoding, parameterised queries, rate limiting, webhook signature verification, secure file-upload handling.
Pipeline security	SAST, dependency (SCA) and secret scanning on every merge; DAST on Staging; container/image scanning where containers are used.
Assurance	VAPT before each go-live (launch surface for R1; targeted tests for identity R2, refunds R3 and payouts R6; full scope before Phase 2A completion in R6), with no critical or high findings open at release (SOW §14.1).

7.2 Data privacy — DPDP Act

- **Notice & consent:** clear notices at registration and for optional processing (e.g. marketing and WhatsApp opt-in), with consent recorded and withdrawable.
- **Purpose & minimisation:** collect only what each role needs; KYC fields limited to what payouts and verification require.
- **Data principal rights:** processes for access, correction, erasure and grievance. In-app account deletion also meets app-store requirements.
- **Retention:** defined per data class, with financial records retained as law requires and other personal data deleted or anonymised when no longer needed.
- **Security safeguards & breach readiness:** the controls in 7.1, plus an incident process that lets SportSeek meet its notification obligations under the DPDP Act, 2023 and the DPDP Rules as notified.
- **No production personal data in non-production environments.**

7.3 Performance, availability and resilience

- **Targets:** API p95 latency, peak concurrency and uptime (e.g. 99.5%, SOW §6.2) are set by SportSeek (D20) and written into the HLD as testable numbers.
- **Load testing** every release on a production-like Staging environment, covering search, availability, booking checkout, registration and ledger queries.
- **Design for load:** indexed geo-queries for venue search, caching of read-heavy reference data, pagination everywhere, asynchronous processing for notifications and reconciliation.
- **Resilience:** timeouts, retries with back-off and circuit breakers on every integration; graceful degradation (e.g. booking continues if a notification channel is down, with messages queued).
- **Recovery:** automated backups with point-in-time recovery; restore rehearsed before launch; documented recovery objectives agreed with SportSeek.

7.4 Observability

Structured logs with correlation IDs, application metrics, distributed tracing (OpenTelemetry-compatible), error tracking, uptime checks and business health dashboards such as bookings/min, payment success rate, refund backlog, notification delivery rate and ledger sync lag. Alerts go to the on-call rota with runbook links. All of it runs in SportSeek's accounts from day one (SOW §6.2).

8. Engineering practices and standards

Practice	How we work
Source control	SportSeek-owned repositories; trunk-based development with short-lived branches; protected main branch; clean, meaningful commit history (SOW §13).
Code review	Every change peer-reviewed; a checklist covering security, tests, observability and docs; SportSeek developers review and are reviewed.
Testing pyramid	A broad unit-test base; API and contract tests at module boundaries; a focused set of end-to-end journeys for mobile and web; coverage gates on new code.
CI/CD pipeline	Build → unit/API/contract tests → static analysis & security scans → package → deploy to Dev → automated regression → promote the same artefact to Staging → approval → Production. Build once, promote everywhere.
Infrastructure as code	All environments defined as code in SportSeek's cloud accounts; no manual changes; reviewed like application code.
Database migrations	Versioned, automated, backward-compatible (expand → migrate → contract); every migration has a tested rollback or roll-forward plan.
Feature flags	Decouple deploy from release; staged enablement of new partner types, waitlist, WhatsApp; instant kill-switch.
API standards	REST with OpenAPI specifications; consistent errors, pagination and idempotency keys; versioning policy; Postman collections published (SOW §13).
Mobile release engineering	Automated builds and signing; phased store rollouts; crash and performance monitoring; minimum-version enforcement.
Documentation	HLD, LLD, ADRs, ERDs, runbooks and API docs kept as code next to the system and updated as part of Definition of Done. Architecture diagrams reflect the actual build (SOW §13).

9. AI-accelerated engineering

Speed comes from putting AI agents into every engineering step while keeping senior engineers accountable for every outcome. This is how a pod of about 10 people ships two production features a month on a live payments platform.

9.1 Spec-driven development

- 1. Specify:** the analyst and architect, working with AI, produce a feature spec: user stories, acceptance criteria, API contract (OpenAPI), data changes, UI frames, and security notes for sensitive areas. The PO approves it.
- 2. Ground:** the repository holds the context the agents need. That means architecture rules, module boundaries, coding conventions, the ADRs, and examples of approved patterns (e.g. how idempotent payment commands are written). Agents follow the codebase's standards rather than inventing their own.
- 3. Generate:** coding agents implement the spec across .NET, PostgreSQL migrations, React Native and React. They generate unit and API tests alongside the code and run the build and tests locally.
- 4. Review & harden:** a senior engineer reviews the change, fixes design issues and edge cases, and owns the merge. AI provides a first-pass review; security-sensitive code gets a second human reviewer.
- 5. Verify:** CI runs the full gates: tests, coverage, architecture tests (module boundaries), SAST, dependency and secret scanning. QA extends the AI-generated regression and E2E suites.
- 6. Document:** API docs, Postman collections, ERDs and runbook sections are regenerated from the code and checked in the same pull request.

9.2 Where AI makes the biggest difference on this programme

Area	AI contribution	Human responsibility
Phase 1 understanding	Maps the live .NET / React Native / PostgreSQL code in days: architecture, ERD, API catalogue, risk hotspots	Architect validates and decides what to trust
Web application (F6, F10)	Ports existing mobile journeys to React/Next.js against the same APIs, reusing design tokens and generating parity checks	Web engineer owns UX quality, SEO, accessibility
Regression safety net	Generates and extends suites for every live journey, so the monthly train stays safe as scope grows	QA curates, removes flaky tests, adds exploratory tests
Identity & data migration (F3)	Drafts duplicate-detection queries, merge scripts and reconciliation reports	Engineers verify on Testcontainers and Staging; SportSeek approves merge rules
Integrations	Scaffolds adapters for Razorpay, SMS, WhatsApp BSP, FCM/APNs and email from provider documentation	Engineers validate against sandboxes; webhook security reviewed by the architect
Documentation & KT	Keeps HLD/LLD, API docs and runbooks current; answers questions over the codebase	Architect reviews; SportSeek signs KT checklist

9.3 Guardrails

- Enterprise-terms AI tools only, approved by SportSeek; no training on SportSeek code or data.
- No production data or secrets in agent context; agents have no production access and cannot deploy.
- Every merge has a named human reviewer who can explain the change; two reviewers for identity, payments, refunds, payouts and KYC.
- Same quality and security gates for all code, however it was produced; generated dependencies are licence-checked.
- AI assistance is recorded in pull requests and reported monthly (share of AI-assisted changes, review time, rework rate).

10. Technical plan for the 20 October 2026 launch

For launch we deliberately avoid changes to identity, payments or the data model. The work makes the existing system safe, visible and recoverable under public load, and adds two back-end-led features that carry little risk: **F1 Email notifications** for all live alert triggers and **F2 Booking & event reminders**. AI compresses the work into the 16 available working days. It reads the codebase for the assessment and generates a full regression suite for every live journey in days.

Workstream	Technical actions	Done when
Stability	AI-assisted triage of the defect backlog; fix critical/high issues in launch journeys; AI-generated regression suite for all live journeys	No open S1/S2 in launch scope; suite green
F1 / F2	Email channel adapter and templates for existing triggers; reminder scheduler (T-24h, T-2h) with cancellation on booking change	Behind flags; enabled after UAT
Security	SAST, dependency, secret and DAST scans; VAPT of the launch surface; fix critical/high; harden headers, rate limits and OTP abuse controls	VAPT retest clean for critical/high
Performance	Load test the key journeys at SportSeek's launch concurrency (D5); tune queries, indexes and scaling settings	Targets met with headroom
Observability	Central logs, error tracking, uptime checks, dashboards and alerting with on-call routing	Dashboards and alerts verified in a drill
Recoverability	Verify backups and a timed restore; rollback plan for the release	Restore and rollback rehearsed on Staging
Store & privacy	Check store policies (privacy labels, permissions, in-app account deletion); publish privacy policy and DPDP notice	Submissions accepted or on track
Payments sanity	End-to-end payment, webhook and failure-path verification against the live gateway configuration	Test transactions reconciled

11. Data migration approach

Migration	Approach	Controls
Identity consolidation (R3)	Detect duplicates on verified phone/email; apply SportSeek-approved merge rules; re-point bookings, events and partner records; keep a mapping table for audit	Dry-runs on Staging; record counts reconciled; rollback script
Facilities → generalised services (R4)	Create the "Facility" type; migrate records and amenities; dual-read behind a feature flag until verified	Full facility regression; zero change in partner experience
Historical events import (R6)	CSV templates; validation and dry-run reports; idempotent loads (SOW §9.3)	Import reports signed off by SportSeek
Ledger back-fill (R5)	Back-fill historical gateway transactions from gateway APIs and reports	Totals reconciled to gateway reports

12. Key technical decisions (ADR backlog)

These decisions are made in the HLD with SportSeek. Each ADR records the context, the options, the decision and its consequences.

ADR	Decision	Options considered	Our leaning / criteria	Decide by
01	Identity provider	ASP.NET Core Identity + OpenIddict; Keycloak; managed identity service	OpenIddict within the .NET estate (subject to assessment)	23 Oct 2026
02	Message broker for event bus	Managed queue/stream service in SportSeek's cloud vs self-managed	Managed service, with a transactional outbox either way	23 Oct 2026
03	Service-type attribute storage	PostgreSQL JSONB + JSON Schema; entity-attribute-value; table per type	JSONB + schema validation, mapped through EF Core	23 Oct 2026
04	Notification providers	Existing SMS; WhatsApp BSP; Email provider; FCM/APNs	SportSeek-contracted providers (SOW §15)	23 Oct 2026
05	Web framework	React + Next.js; React Native for Web; React SPA	React + Next.js (SEO, shared React/TypeScript skills)	23 Oct 2026
06	Feature-flag mechanism	Library + config in DB; managed flag service	Simple and auditable; SportSeek-owned	23 Oct 2026
07	API versioning & min-version policy	URI vs header versioning; forced vs soft upgrade	Explicit versions; minimum-version check at app start	23 Oct 2026

13. Technical risks and mitigations

Risk	Mitigation
Phase 1 code quality, or .NET / React Native versions near end of support	Assessment first; upgrade needs raised as ADRs with evidence; framework upgrades scheduled outside launch freeze; change control if scope shifts
Duplicate identities and ambiguous merges	OTP-verified linking; approved merge rules; staged dry-runs; audit mapping table
Route capability or KYC flows differ for new partner types	Confirm with Razorpay by 6 Nov (D10); sandbox proof-of-concept in S1
Regression in live booking/payment flows	Automated regression as a merge gate; feature flags; canary/phased rollouts
Notification template approvals (DLT, WhatsApp) delay go-live	Submit templates early (D12, D13); channel fallback; flag-controlled enablement
Old mobile clients break on new APIs	Versioned APIs; compatibility tests; minimum-version enforcement

14. Why Srivin Platforms

SportSeek needs a partner that can do four things at once: launch safely in weeks, ship new value every month, re-foundation a live platform without breaking it, and leave SportSeek stronger and more independent at the end. This approach is built for exactly that combination:

What SportSeek needs	What Srivin brings in this proposal
A public launch on 20 Oct 2026 that does not backfire	A launch plan with a Scope Gate, code freeze, VAPT, load test, rehearsed rollback and a Go/No-Go that SportSeek controls. Honest about what fits the date.
Identity unified without duplicates or takeovers	An account-linking design with ownership proof, reconciliation of existing data, and an automated acceptance test. It goes beyond the SOW's requirement to detect matches.
Payments that stay compliance-light	Gateway-native ledger with reconciliation, Route split settlement as a hard constraint, PCI scope kept minimal, and separate modelling of SportSeek's own fee revenue.
A platform that grows into 2B/2C	A service-type model that makes expert booking an extension; a domain-event taxonomy that feeds analytics; a modular monolith with extraction paths.
Ownership and independence	Everything in SportSeek's accounts; embedded SportSeek developers; KT every milestone with shadow and reverse-shadow; documentation as code.
Faster go-to-market	An AI-native pod that ships two features to production every month from launch day, and completes Phase 2A on 23 Mar 2027 with less than half the effort of a conventional team.
Predictability	Fixed monthly release dates, dated dependencies, and weekly working software (see the WBS & Integrated Project Plan and the Project Delivery Model).

Srivin credentials — to be completed by Srivin Platforms before issue

[Insert relevant case studies, platform references, team profiles of named key personnel and any certifications. Only verified, client-approved references should be included.]

Appendix A. Glossary

Term	Meaning
ADR	Architecture Decision Record: a short document recording a significant technical decision and its rationale
BFF	Backend-for-Frontend: a thin API layer shaped for a particular client (mobile, web)
BSP	Business Solution Provider for the WhatsApp Business Platform
DLT	Distributed Ledger Technology platform used in India to register SMS senders and templates
DPDP Act	India's Digital Personal Data Protection Act, 2023
HLD / LLD	High-Level Design / Low-Level Design
KT	Knowledge Transfer
OIDC / PKCE	OpenID Connect (identity layer on OAuth 2.0) / Proof Key for Code Exchange (secures public-client logins)
Outbox pattern	Writing events to a table in the same transaction as the business change, then relaying them to the broker, so events are never lost or sent for rolled-back work
Route	Razorpay's split-settlement product for routing funds to linked partner accounts
SIT / UAT	System Integration Testing / User Acceptance Testing
VAPT	Vulnerability Assessment and Penetration Testing